

ATNLP Exam Guide 中文速记讲义

按 Exam Guide Topics 整理

Generated for revision

2026-05-20

Contents

使用方式	2
1. Machine Translation	3
2. Question Answering	4
3. Data and Synthetic Data	5
4. Evaluation	6
5. Long Context Modelling	7
6. RAG	8
7. Multilingual LLMs	9
8. Bias and Ethics	10
9. Safety, Watermarking and Adversarial Attacks	11
10. Instruction Finetuning, RLHF and Basic RL	12
11. Summarisation	13
12. Parameter-Efficient Finetuning	14
13. Distillation	15
14. Reasoning During Decoding	16
15. Mechanistic Interpretability	17
16. Syntactic Parsing	18
17. Perceptually-Grounded NLP	19
18. Vision-Language Models	20
19. Text-Only AI Agents	21
20. Vision-Based Agents and OpenCUA	22
最后总速记	23

使用方式

这份讲义按 exam guide 的 topic 顺序整理。每个 topic 都压成五块：一句话主线、必背知识点、常考对比/公式/流程、易错点、30 秒答题模板。考前优先背每节的“主线”和“模板”，再用“对比/公式/流程”补细节。

1. Machine Translation

一句话主线：机器翻译是在歧义、形态、语序和数据不均衡的限制下，用平行语料训练 encoder-decoder 或 LLM 系统，并用人工与自动评价判断译文是否忠实、流畅。

必背知识点

- MT 难点一是词义歧义，例如 bank 需要上下文决定是银行还是河岸。
- MT 难点二是形态复杂，形态丰富语言中一个词可能包含词根、数、格、时态等多层信息。
- MT 难点三是语序差异，不同语言有 SVO、VSO、后置词、形容词位置差异等。
- 平行语料是传统 MT 和 NMT 的核心数据，来源包括 Bible、Hansard、Europarl、UN、网页爬取语料。
- Web 平行数据抽取流程：language identification、cleaning、document alignment、sentence splitting、sentence alignment、deduplication、filtering。
- 低资源语言常遇到网页少、语言识别差、文本质量低、句子对齐噪声大。
- NMT 学习 $p(T|S)$ ，输出同时依赖完整源句和已生成目标历史。
- 标准 MT 架构是 encoder-decoder Transformer：encoder 编码源句，decoder 通过 cross-attention 访问源句表示并自回归生成目标句。
- 人工评价看 adequacy 和 fluency；自动评价用于低成本比较系统迭代。
- LLM 翻译进入 pretrain-prompt 范式，但低资源、领域迁移、评价可靠性仍不是 solved problem。

常考对比/公式/流程

- 翻译目标： $T^* = \operatorname{argmax}_T p(T|S)$ 。
- 序列分解： $p(T|S) = \prod_i p(y_i | y_{<i}, x_1, \dots, x_m)$ 。
- Phrase-based MT vs NMT：前者依赖人工特征、稀疏 log-linear model 和复杂搜索；后者端到端、连续表示、条件化于全源句。
- Adequacy：是否保留源句意义；Fluency：目标语是否自然、合语法、用词恰当。
- ChrF：字符 n-gram precision/recall 的 F-score，常比只看词级 overlap 的 BLEU 更适合形态丰富语言。
- COMET：用跨语言预训练模型，并在 WMT 人类评价数据上 fine-tune，更贴近人工判断，但仍有泛化限制。

易错点

- 自动指标高不等于译文一定好；自动指标可能漏掉语义错误、遗漏或添加。
- 不要只说 MT 数据来自网页；考试要写抽取、对齐、去重、过滤的 pipeline。
- Decoder 不是只看目标历史，它还通过 cross-attention 看源句。
- BLEU/ChrF/COMET 不是同一类：BLEU/ChrF 偏 overlap，COMET 是 learned metric。

30 秒答题模板

Machine translation 是给定源句 S ，寻找最可能目标句 T^* 的任务，即最大化 $p(T|S)$ 。它困难在词义歧义、形态复杂、语序差异和低资源数据不足。现代 NMT 通常用 encoder-decoder Transformer：encoder 表示源句，decoder 在目标历史和 cross-attention 条件下逐词生成。训练主要依赖平行语料，网页数据需经过语言识别、清洗、文档/句子对齐、去重和过滤。评价上，人工 adequacy/fluency 仍是 gold standard；ChrF、BLEU、COMET 等自动指标适合大规模比较，但不能完全替代人工判断。

2. Question Answering

一句话主线：QA 从“给定文本找答案”的 reading comprehension，发展到“先检索证据再生成答案”的 open-domain QA，核心始终是问题、证据和答案之间的对齐。

必背知识点

- QA 是自动回答自然语言问题的系统，是测试语言理解的重要 test bed。
- Reading Comprehension 形式是 $P, Q \rightarrow A$ ：给定 passage 和 question，输出 answer。
- Extractive QA 要求答案是 context 中连续 span，常预测 start/end indices。
- SQuAD 来自 Wikipedia passages 和众包问题，答案是 passage 中短 span。
- SQuAD 常用 Exact Match 和 token-level F1 评价。
- SQuAD 局限是设置干净、passage 短、答案多为 span，缺少真实检索、歧义和不可回答问题。
- Natural Questions 使用真实 Google queries 和完整 Wikipedia 页面，更接近真实搜索 QA。
- Open-domain QA 不给定 passage，需要先从大规模文档库检索 evidence。
- Retriever-reader pipeline：retriever 找 top-k passages，reader/generator 抽取或生成答案。
- Fusion-in-Decoder 将多个 retrieved passages 分别编码，在 decoder 中融合证据生成答案。

常考对比/流程

- RC vs ODQA：RC 给定 passage；ODQA 不给 passage，要先检索。
- SQuAD vs Natural Questions：SQuAD 更干净、短 passage、span answer；NQ 更真实、完整页面、长/短答案、无答案和歧义。
- Sparse retrieval vs dense retrieval：前者 BM25/TF-IDF 词面匹配；后者 dense vectors 和 dot product 做语义匹配。
- Extractive QA 输出约束： $1 \leq \text{start} \leq \text{end} \leq n$ 。
- FiD 流程：检索 top-k passages $\rightarrow$ 每个 question + passage 独立 encoder $\rightarrow$ decoder attends over all representations $\rightarrow$ 生成答案。

易错点

- SQuAD 上超过人类不等于 QA 被解决，因为真实 QA 还有检索、多证据和开放生成问题。
- EM 很严格；语义正确但表达不同也可能 0 分。F1 给 token overlap 的部分分。
- ODQA 的错误可能来自 retriever，也可能来自 reader/generator。
- FiD 的“fusion”发生在 decoder，不是在 encoder 里把所有 passages 粗暴拼接成一段。

30 秒答题模板

Question Answering 要求系统理解问题、定位证据并输出答案。Reading Comprehension 中输入是 passage P 和 question Q，输出 answer A；extractive QA 通常预测答案 span 的 start/end。SQuAD 推动了 neural RC，但它缺少真实检索和无答案问题。Natural Questions 更真实，因为问题来自 Google queries，文档是完整 Wikipedia 页面。Open-domain QA 更难，因为系统必须先检索 top-k evidence，再由 reader 或 generator 回答。Fusion-in-Decoder 是生成式 ODQA 方法：多个 retrieved passages 分别编码，decoder 在生成时融合所有证据。

3. Data and Synthetic Data

一句话主线：LLM 能力很大程度由数据决定，关键不是“越多越好”，而是来源、筛选、去重、混合、合成与伦理共同决定训练信号质量。

必背知识点

- 数据阶段：pre-training data 学通用语言和知识；mid/annealing data 强化高质量能力；post-training data 学指令跟随、偏好和安全。
- 主要来源：Wikipedia 质量高但代表性有限；Common Crawl 规模大但噪声、重复、版权和地域偏差严重；代码数据要看 license。
- Data selection 目标：提升性能、降低训练成本、保护 evaluation integrity、减少 toxic/bias/PII 风险。
- 常见过滤：language filtering -> heuristic filtering -> quality/classifier filtering -> domain filtering -> deduplication -> toxic/PII filtering -> data mixing。
- Deduplication：exact dedup 去完全重复；approximate dedup 去近重复；MinHash 用 Jaccard signature 高效找重复。
- FineWeb 从 Common Crawl 构建高质量 web corpus，核心思想是 quality trumps quantity。
- FineWeb-Edu 用强 LLM 生成教育价值 synthetic labels，再训练轻量分类器筛出教育内容。
- Synthetic instruction data 是模型、程序或自动流程生成 instruction-response pairs，用于 instruction tuning 或 distillation。
- 合成方法包括 stronger model distillation、self-generation + verification、Evol-Instruct、procedural generation。
- 数据伦理包括版权、PII、toxicity、方言误伤、群体代表性不足和强势文化过度代表。

常考对比/流程

- Pre-training vs post-training：前者学语言/知识分布，后者学“如何按人类意图回答”。
- Heuristic filtering vs classifier filtering：前者便宜快但浅；后者更能捕捉质量但依赖训练标签。
- Exact vs approximate dedup：完全相同 vs 模板/轻微改写近似相同。
- MinHash 流程：文档转 token/n-gram 集合 -> 多个 hash 得签名 -> 签名匹配比例近似 Jaccard -> 删除近重复簇。
- Synthetic data 优势是低成本扩展任务覆盖；风险是继承 teacher 错误、模式坍塌、放大偏见。

易错点

- 不要把“更多数据”当成必然更好；低质量重复数据会浪费算力并伤害泛化。
- 去 toxic 内容不是删得越多越好，可能误伤边缘群体语言和讨论歧视的文本。
- PII 处理要区分公开知识和隐私数据。
- FineWeb-Edu 不是纯 synthetic corpus，而是用 synthetic labels 做真实数据筛选。

30 秒答题模板

Data 是 LLM 的核心训练信号，决定知识、语言模式、偏差和安全行为。训练数据来自 Wikipedia、Web/Common Crawl、代码等，但必须经过 language filtering、heuristics、quality filtering、deduplication、toxicity/PII filtering 和 data mixing。FineWeb 展示了高质量筛选和去重比盲目扩大规模更重要；FineWeb-Edu 用 LLM 生成教育价值标签来筛选高质量教育数据。Synthetic instruction data 可低成本扩展 instruction tuning，但需要验证、多样性控制和 human data 混合，同时注意版权、隐私和代表性伦理问题。

4. Evaluation

一句话主线：LLM evaluation 没有单一真指标，必须按 use case、答案类型、ground truth、metric、prompt 和部署风险做多维评价。

必背知识点

- Evaluation 目标：比较系统、判断改动是否有效、评估是否适合部署。
- Evaluation framework 维度：answer type、ground truth、metric、prompt formulation、deployment criteria。
- Intrinsic evaluation 直接评模型内部任务，如 perplexity、cloze；便宜但不能代表事实性、推理和 alignment。
- Task-based benchmarks 如 MMLU、MMLU-Pro、GPQA、HLE 用于标准化比较。
- Benchmark saturation：模型接近满分后，benchmark 失去区分能力。
- Contamination：测试集进入训练数据，导致模型靠记忆得高分，破坏泛化评价。
- LM Arena：真实用户比较两个匿名模型回答，用 pairwise preference 聚合开放式回答质量。
- Elo：表现高于预期就涨分，低于预期就降分，适合 pairwise comparison。
- LLM-as-a-Judge：用强模型评估输出，可扩展、有解释，但有位置偏差、自我偏好、prompt sensitivity。
- 真实部署评价要看单例错误、失败模式、风险严重性，不只看平均分。

常考对比/公式

- Intrinsic vs task-based：前者测语言建模等内部能力；后者测具体任务表现。
- Closed-ended vs open-ended：有标准答案可 accuracy/F1；开放生成常用 human/LLM judge、rubric、pairwise preference。
- Perplexity 越低表示模型对下一个 token 越确定，但必须在同一数据集/tokenization 下比较。
- Elo 更新： $R_A' = R_A + K(O_A - E_A)$ ； O_A 是实际结果， E_A 是预期胜率。
- Contamination 检测：train-test overlap 报告、private/dynamic test sets、public-private 分数差、prefix completion 记忆测试。

易错点

- 低 perplexity 不等于事实正确、推理强或用户喜欢。
- Leaderboard 高分不等于真实部署可靠，尤其安全关键场景不能只看平均分。
- LLM judge 不是客观真理，需要人类校准和偏差控制。
- Saturation 和 contamination 会一起让公开榜单越来越不可信。

30 秒答题模板

评价 LLM 难在输出开放、没有唯一 ground truth、能力多维且 prompt sensitive。应先明确 use case 和风险，再选答案类型、metric、prompt 和部署标准。Perplexity 等 intrinsic 指标适合追踪语言建模，但不能测事实、推理和 alignment；MMLU/GPQA/HLE 等 benchmark 可标准化比较，但有 saturation 和 contamination。开放式任务可用 LM Arena 的人类 pairwise preference 和 Elo 聚合，或 LLM-as-a-Judge，但要注意 judge bias 和人类校准。

5. Long Context Modelling

一句话主线：长上下文建模要同时解决位置泛化、二次方 attention 成本和“能放进去不等于能用好”的长程推理问题。

必背知识点

- Long context 指模型处理数万到百万 tokens，用于长文档、代码库、法律材料、agent 历史等。
- 三大挑战：training data mismatch、quadratic attention cost、long-range reasoning difficulty。
- Transformer 需要 positional encoding，因为 self-attention 本身没有顺序信息。
- Sinusoidal PE 用不同频率的 sin/cos 给位置固定 fingerprint；低频表全局，高频表局部。
- RoPE 旋转 Q/K 向量编码位置，使相对位置通过旋转角度差进入 attention score。
- RoPE 外推问题：远位置高频旋转会 wrap around，导致相对距离噪声化。
- Position Interpolation 把长位置压缩回训练长度范围，如 4096 映射到 2048 范围。
- YaRN 对不同波长维度分段插值，兼顾局部细节和全局位置。
- Efficient attention 包括 sliding window、dilated window、global tokens、KV cache、KV compression。
- Evaluation 包括 Needle in the Haystack、LongBench、lost-in-the-middle。

常考对比/公式

- 输入表示： $z_{pos} = x_{pos} + PE_{pos}$ 。
- Attention: $Attention(Q, K, V) = \text{softmax}(QK^T / \sqrt{d})V$ 。
- Full attention 复杂度： $O(n^2)$ ，因为每个 token pair 都要比较。
- Sinusoidal vs RoPE：前者把位置向量加到 token embedding；后者旋转 Q/K，让相对距离体现在 attention score。
- PI vs NTK vs YaRN：PI 压缩位置索引；NTK 调整 RoPE 频率；YaRN 分段处理不同波长。
- Needle test 变量：context length 和 needle depth。

易错点

- 长 context window 不等于 effective long-range reasoning。
- Needle test 只是检索 sanity check，不等于真实长文档理解。
- Sparse attention 提升效率但牺牲部分全局信息整合。
- Lost-in-the-middle 说明模型常更会用开头/结尾，忽略中间证据。

30 秒答题模板

Long-context modelling 让模型处理很长输入，但核心挑战是训练数据多为短文档、full attention 是 $O(n^2)$ 、以及长窗口不等于能推理。位置上，sinusoidal PE 用 sin/cos 编码位置，RoPE 通过旋转 Q/K 让 attention 感知相对距离；扩展 RoPE 可用 Position Interpolation、NTK scaling、YaRN。效率上用 sparse/sliding/global attention、KV cache 或 compression。评价要看 Needle、LongBench 和 lost-in-the-middle，判断模型是否真的能找到并使用远处信息。

6. RAG

一句话主线：RAG 把 LLM 参数记忆和外部可更新检索记忆结合，核心流程是先检索证据，再基于证据生成，并尽量可追溯。

必背知识点

- RAG = Retrieval-Augmented Generation，适合事实、最新知识、长尾实体、领域知识和引用需求。
- Retriever-reader 架构：retriever 从 corpus/index 找 top-k passages，reader/generator 基于证据回答。
- Parametric memory 是模型参数中的知识；non-parametric memory 是外部文档库、数据库、知识图谱。
- Sparse retrieval 如 TF-IDF/BM25，靠词项重叠，解释性强，适合精确实体。
- Dense retrieval 用 bi-encoder 把 query/passages 编成向量，用 dot product/cosine 检索，强于语义匹配。
- Efficient retrieval 用 MIPS、FAISS、ANN、IVF/k-means 避免逐一比较海量向量。
- RAG-Sequence 整段输出用同一组文档；RAG-Token 不同 token 可依赖不同文档。
- Attribution 指答案声明要能被引用证据支持；有引用不等于 faithful。
- GraphRAG 把文档组织成实体、关系、社区/图结构，适合多跳、企业文档和全局摘要。
- Agentic RAG 让 agent 动态决定查询、检索、改写、验证和多步工具调用。

常考对比/流程

- Open-domain QA vs reading comprehension：前者先检索再回答；后者通常已给 passage。
- Sparse vs dense：词面匹配、可解释、便宜 vs 语义匹配、依赖训练和向量索引。
- Bi-encoder vs cross-encoder：前者可预索引、高效初筛；后者联合编码 query-doc，精度高但慢。
- RAG 流程：query -> retrieve top-k -> optional rerank -> generator/reader answer -> attribution/verification。
- RAG vs long context：RAG 先选相关信息、成本低但依赖检索；long context 放更多内容但可能信息过载和 lost-in-the-middle。

易错点

- RAG 不能完全消除 hallucination，检索错、排序错、证据不足、生成器忽略证据都会失败。
- 有 citation 不代表答案正确，引用可能不相关或只支持部分说法。
- Dense retrieval 不一定总比 sparse 好，精确实体和关键词场景 sparse 很强。
- GraphRAG 的代价是构图、实体关系抽取错误和系统复杂度。

30 秒答题模板

RAG 通过 non-parametric memory 补充 LLM 的 parametric memory：系统先从外部 corpus 检索 top-k 证据，再由 reader/generator 生成答案，并最好提供 attribution。Retriever 可用 sparse 方法如 TF-IDF/BM25，也可用 dense bi-encoder 和 FAISS/MIPS 做语义检索；cross-encoder 常用于 reranking。RAG 能提升事实性、更新性和长尾知识，但不能完全避免 hallucination。变体包括 RAG-Sequence/RAG-Token、GraphRAG 和 Agentic RAG。

7. Multilingual LLMs

一句话主线：多语言 LLM 用共享模型服务多语言用户，但必须处理数据不均衡、tokenizer 不公平、transfer/interference、评测和责任问题。

必背知识点

- 多语言建模意义：公平、可及性、非英语知识覆盖、跨语言迁移。
- 核心挑战：高低资源语言不均衡、tokenization 不公平、positive transfer/negative interference、评测难、安全文化差异。
- FineWeb2 是面向 Common Crawl 的多语言高质量 web corpus pipeline。
- FineWeb2 包括语言识别、tokenisation、去重、质量过滤、rehydration 等步骤。
- Rehydration 利用网页在多个 crawl snapshots 中出现情况帮助判断质量。
- Data mix 决定不同语言采样比例，影响高资源效率与低资源曝光。
- Temperature sampling 在 proportional 和 uniform 之间折中。
- Tokenization fertility 高的语言会消耗更多 token、上下文和计算预算。
- ATLAS scaling laws 强调目标语言表现不仅取决于本语言数据，也取决于相似语言带来的 transfer 和 interference。
- Responsible multilingual modelling 要关注 safety transfer、local harms、data sovereignty、participatory design 和语言社区参与。

常考对比/公式

- Positive transfer vs negative interference：一种语言帮助另一种语言 vs 一种语言训练损害另一种语言。
- Curse of multilinguality：固定容量/预算被更多语言共享，单语言表现可能下降。
- Proportional sampling 偏高资源；uniform sampling 平衡语言但会过采样低资源；temperature sampling 折中。
- Temperature sampling: $p_i = n_i^\alpha / \sum_j n_j^\alpha$ ； α 越小越接近 uniform，越大越接近 proportional。
- Culturally adapted benchmark vs translated benchmark：前者按本地文化重写；后者简单翻译，可能不自然、不等价、英语中心。

易错点

- 英语强不等于 multilingual strong。
- 平均分高可能掩盖低资源语言严重失败。
- Tokenizer 不是中立工程细节，它决定不同语言使用模型预算的“价格”。
- “Think in English then answer target language”不是根本解决方案，可能丢失本地语境。
- 低资源语言数据少不代表语言价值低，而是数字资源和权力不平等。

30 秒答题模板

Multilingual LLM 目标是一个模型覆盖多语言，同时利用共享参数产生跨语言迁移。主要挑战包括数据不均衡、tokenization 不公平、positive transfer 与 negative interference、评测和安全文化差异。FineWeb2 通过语言识别、去重、质量过滤和 rehydration 构建多语言 web data；data mix 可用 temperature sampling $p_i = n_i^\alpha / \sum n_j^\alpha$ 在高资源效率和低资源曝光间折中。ATLAS scaling laws 强调语言间 transfer matrix。评价时不能只看平均分，要分语言报告、专家验证、文化适配，并考虑 data sovereignty 和社区参与。

8. Bias and Ethics

一句话主线：AI ethics 不是“模型有没有礼貌”，而是追问 LLM 在数据、训练、部署中如何造成 harm：谁受益、谁受害、谁决定、谁负责。

必背知识点

- LLM 风险分类包括 discrimination/exclusion、information hazards、misinformation、malicious use、HCI harms、environmental/socioeconomic harms。
- Allocational harm 影响资源/机会分配，如招聘、贷款、教育、医疗。
- Representational harm 以刻板、贬低、缺失方式呈现群体，强化偏见。
- Bias measurement 不能只靠一个 metric，要结合 benchmark、counterfactual testing、human evaluation。
- WEAT 用 target sets 与 attribute sets 的 cosine similarity 差测 embedding association。
- BBQ 用 ambiguous context 测 stereotype reliance，用 disambiguated context 测 bias 是否压过正确证据。
- Counterfactual testing 只替换姓名、性别、族裔等人口统计信号，看输出是否系统性变化。
- Mitigation 包括 preprocessing、in-training、intra-processing、post-processing。
- RLHF 可减少显性有害行为，但 reward model 会编码标注者文化视角。
- 技术修复必须配合部署审计、社区反馈、制度问责。

常考对比/流程

- Allocational vs representational：前者影响现实机会，后者影响群体形象。
- Measurement vs mitigation：测量发现/评估偏见，缓解减少 harm；指标改善不等于真实世界 harm 消失。
- WEAT 直觉：比较词 w 与属性集 X/Y 的平均 cosine similarity 差。
- Good practice：structured benchmark + domain-specific counterfactual tests + open-ended human evaluation。

易错点

- 不要把 bias 简化成“训练数据脏”；部署场景、标注者、制度结构都会制造 harm。
- 不要说 WEAT 能直接预测真实部署偏见；它主要测 embedding association。
- 不要把 RLHF 当万能 debiasing；它可能留下 subtle bias。

30 秒答题模板

LLM ethics 关注系统生命周期中的 harm 和责任。Bias 主要造成 allocational harm 与 representational harm，可用 WEAT、BBQ、counterfactual testing 等测量，但单一指标不够。Mitigation 可在数据、训练、推理、输出阶段做，但因为 bias 上下文相关且会 whack-a-mole，所以还需要审计、反馈和问责机制。

9. Safety, Watermarking and Adversarial Attacks

一句话主线：LLM safety 关注避免伤害，security 关注防止被攻击或绕过；jailbreak 常是 security failure 导致 safety failure。

必背知识点

- Safety 包括有害内容、偏见、错误信息、隐私泄露、危险建议。
- Security 包括 prompt injection、jailbreak、data exfiltration、滥用、检测规避。
- Watermarking 在生成时嵌入统计可检测模式，用于识别 AI-generated text。
- 文本水印不能像图片水印一样嵌不可见标记，只能改变 token 概率偏好。
- Green list/red list 由上下文和 secret key 决定，提高 green token 采样概率。
- 检测时统计 green token 比例是否显著高于无水印期望。
- 水印局限：短文本弱、改写/翻译/摘要破坏信号、key 泄露可伪造/规避。
- Illicit content attack 诱导模型生成违法、危险或平台禁止内容。
- Adversarial suffix attack 优化一段 suffix，让模型从拒绝转为服从。
- 防御需要多层：数据过滤、对齐、分类器、拒绝策略、红队、监控、水印。

常考流程/对比

- Watermark 流程：secret key + context 划分 green/red tokens -> 给 green tokens 加 logit bias -> 生成文本 -> detector 做统计检验。
- False positive：无水印文本被判水印；false negative：有水印文本没检测出。
- Adversarial suffix：positive affirmation objective 让模型倾向“Sure, here is...”；coordinate gradient search 在离散 token 上搜索 suffix。
- Closed model 更适合水印控制；open-source model 容易被改 decoding 或移除水印。

易错点

- Watermark 只能辅助来源识别，不能阻止有害内容。
- 训练数据过滤不能删掉所有 dual-use 知识，否则会伤害医学、教育、安全研究等合法用途。
- Blacklist 一个 suffix 不够，攻击者可重新优化新 suffix。

30 秒答题模板

Safety 是避免伤害，security 是防止系统被攻击或滥用。Watermarking 在 decoding 时用 key 把 token 分成 green/red list，提高 green token 概率，检测时看统计显著性；但短文本、改写和 key 泄露都会削弱它。Adversarial suffix attack 用优化搜索触发模型服从恶意请求，说明安全对齐可能依赖脆弱表面模式，因此需要多层防御和红队测试。

10. Instruction Finetuning, RLHF and Basic RL

一句话主线：先用 SFT 教模型按指令和格式输出，再用偏好或可验证 rewards 优化行为；coursework 里体现为 GSM8K 上从 SFT 到 GRPO。

必背知识点

- Instruction finetuning/SFT 用 prompt-completion 数据继续训练模型，让模型学会任务格式和行为。
- SFT 不只是增加知识，也塑造输出格式、指令跟随和 reasoning trace 风格。
- Base vs Instruct：架构可相同，但 Instruct 已经过指令对齐，通常更会遵循用户意图。
- Basic RL：policy 生成 action/output，reward function 给奖励，训练目标是最大化期望奖励。
- RLHF 常见流程：收集偏好数据 -> 训练 reward model -> 用 RL 优化 policy，同时约束不要偏离 reference model。
- Coursework 使用 Qwen-2.5-0.5B/Instruct，在 GSM8K 上做 zero-shot、SFT、GRPO。
- GRPO 不训练 neural reward model，而用 verifiable rewards。
- Format reward 检查输出格式，correctness reward 检查数字答案是否正确。
- GRPO 生成多个候选答案，基于组内相对好坏优化，不需要 critic model。
- Reward hacking 指模型学会拿容易奖励，如格式，而不是真正解决问题。

常考对比/流程

- SFT vs RL：SFT 模仿给定答案；RL 根据 reward 改变生成策略。
- RLHF vs coursework GRPO：RLHF 通常学 reward model；GRPO 用规则/可验证奖励。
- Reference model 约束的作用：防止 policy 为追求 reward 过度偏离原语言能力。
- 典型现象：format reward 快速上升，correctness reward 震荡/停滞，说明模型可能优先优化表面格式。

易错点

- 不要把 SFT 说成“只提高知识”；它也强烈塑造输出格式。
- 不要把 GRPO 等同 PPO；GRPO 用 group-relative comparison，省掉 critic。
- 奖励高不一定代表真实能力强，可能是 reward hacking 或 Goodhart's law。

30 秒答题模板

Instruction finetuning 用指令-回答样本训练模型，让它学会任务格式和 CoT 解题。RLHF 则用人类/教师偏好训练 reward model，再用 RL 优化 policy，并用 reference constraint 防止模型跑偏。Coursework 中 GRPO 用可验证奖励替代 reward model，对同一 prompt 生成多候选并按相对奖励学习；风险是模型可能先学会格式奖励，而不是真正提高数学正确性。

11. Summarisation

一句话主线：摘要不是简单“写短”，而是在任务类型、数据、模型和评价之间权衡，压缩文本同时保留关键信息、连贯性和事实忠实性。

必背知识点

- Summarisation 是把一个或多个长文档压缩成更短、连贯、信息充分的文本。
- 设计摘要系统要考虑四件事：任务类型、训练数据、模型/学习方法、评价方式。
- Single-document 总结一个文档；multi-document 总结多个相关文档。
- Multi-document 更难：冗余、冲突、一致性、长上下文。
- Generic summary 总结全文重点；query-focused summary 只总结与 query 相关内容。
- Extractive 选原文句子，事实更稳；abstractive 生成新表达，更灵活但更易 hallucinate。
- 抽取式摘要可建模为句子级二分类：每句是否进入摘要。
- Pointer-generator 结合生成和复制，适合保留人名、数字、术语。
- Coverage mechanism 记录过去 attention，减少重复生成。
- 摘要评价需看 relevance、fluency、coherence、faithfulness、conciseness。

常考对比/公式

- 抽取式建模：文档 $d=(s_1, \dots, s_n)$ ，预测 $y_i \in \{0,1\}$ 。
- Pointer-generator: $P(w)=p_{\text{gen}} P_{\text{vocab}}(w)+(1-p_{\text{gen}}) \sum_{i:w_i=w} a_i^t$ 。
- ROUGE：基于候选摘要和参考摘要的 n-gram overlap，偏 recall。
- BERTScore：用 contextual embeddings 做 soft overlap，更能处理同义改写。
- CNN/DailyMail vs XSum：前者新闻 highlights，LEAD-3 很强；后者单句极限摘要，更抽象。
- 长文本摘要流程：chunk -> 局部摘要 -> 汇总局部摘要；风险是早期遗漏不可恢复。

易错点

- 不要只答“摘要越短越好”；还要 faithful、informative、coherent。
- ROUGE 高不等于事实正确；词面相似可能掩盖语义错误。
- Extractive 不等于落后，它在医疗、法律等高风险场景很有价值。
- LEAD-3 强说明新闻数据有位置偏置，不一定说明模型真正理解全文。

30 秒答题模板

Summarisation 是把长文本压缩成短而连贯的摘要，并保留核心信息。系统设计要考虑任务类型、数据、模型和评价。任务上可分 single/multi-document、generic/query-focused、extractive/abstractive。Extractive 直接选原句，通常更 faithful；abstractive 生成新表达，更自然但更容易 hallucinate。经典生成模型 pointer-generator 用 p_{gen} 在词表生成和源文复制间切换，并用 coverage 减少重复。评价上 ROUGE 看 n-gram overlap，BERTScore 看语义软匹配，但都不能完全判断事实忠实性。

12. Parameter-Efficient Finetuning

一句话主线：PEFT 冻结大模型，只训练少量参数来适配任务；三大类是改权重、改输入、加模块。

必背知识点

- Full fine-tuning 对 LLM 昂贵：要存参数、梯度、activation、optimizer states。
- PEFT 固定大部分预训练参数 θ ，只更新少量任务参数 ϕ 。
- PEFT 目标是用更低训练成本接近 full fine-tuning 的下游性能。
- 三类 PEFT：parameter composition、input composition、function composition。
- Parameter composition 通过修改/选择/低秩更新原模型参数来适配任务。
- LoRA 冻结 W_0 ，学习低秩矩阵 B, A ，令 $\Delta W = BA$ 。
- LoRA 常用于 attention 的 W_Q, W_K, W_V 等投影矩阵。
- Prompt tuning 学习连续 prompt embeddings，拼到输入前。
- Adapters 在 Transformer 层中插入 down-projection、activation、up-projection 小模块。
- 多任务部署时可共享 base model，只切换 LoRA/adapters/prompt 参数。

常考对比/公式

- Parameter composition: $g_i(x) = f_{\{\theta_i \oplus \phi\}}(x)$ ，例 LoRA。
- Input composition: $g_i(x) = f_{\{\theta_i\}}([\phi_i, x])$ ，例 prompt tuning。
- Function composition: $g_i(x) = f_{\{\theta_i\}} \circ f_{\phi}(x)$ ，例 adapters。
- LoRA: $h = [W_0 + \Delta W]x = [W_0 + BA]x$ 。
- Adapter: $f_{\phi}(x) = W_U \sigma(W_D x)$ ，bottleneck 降低参数量。
- LoRA rank r 控制容量和参数量； r 越大越强但越贵。

易错点

- PEFT 不是不训练；ICL 才是不更新参数。
- Prompt tuning 参数少，但会占上下文并增加推理成本。
- LoRA 可合并进权重，推理效率通常好；adapter 增加额外函数调用。
- Rank 太低或应用层选错会限制 LoRA 表达能力。

30 秒答题模板

PEFT 冻结预训练模型，只训练少量任务参数，解决 full fine-tuning 在 LLM 上显存和存储过高的问题。三类方法是 parameter composition、input composition、function composition，分别对应 LoRA、prompt tuning、adapters。LoRA 用 $\Delta W = BA$ 低秩更新原权重，参数少且可合并；prompt tuning 把适配信息放进输入；adapters 插入 bottleneck 模块，稳定但推理多一步。

13. Distillation

一句话主线：Knowledge distillation 用强 teacher 生成数据、标签、反馈或分布，再训练 student 学到能力，常用于把专有强模型能力转给小/开源模型。

必背知识点

- Teacher 通常更强、更大或专有；student 更小、更便宜、更可控。
- KD 不只是 compression，也用于能力增强、指令跟随、领域适配、自我改进。
- Proprietary LLM 强但黑盒、贵、有隐私风险；open-source 可控但通常弱。
- KD recipe: 选 skill/domain/template -> 准备 seed knowledge -> 生成 distillation knowledge -> 训练 student。
- 六种获取知识: labelling、expansion、data curation、feature、feedback、self-knowledge。
- Labelling: 给定 x ，teacher 生成 y 。
- Expansion: teacher 生成新输入 x 和对应输出 y 。
- Feature: 用 teacher 内部表示/分布，通常需要 white-box。
- Feedback: teacher 评价 student 输出，可生成 preference data。
- 转移方法包括 SFT、divergence/similarity、RL、ranking optimisation/DPO。

常考对比/公式

- KD 数据: $D_{I^{kd}} = \{\text{Parse}(o, s) \mid o \sim p_T(o \mid I \oplus s), s \in S\}$ 。
- Student 目标: $L = \sum_I L_I(D_{I^{kd}}; \theta_S)$ 。
- Black-box KD: 只用输出/评价；white-box KD: 用 hidden states、logits、分布。
- SFT vs divergence: SFT 学离散答案；divergence 拟合 teacher 完整输出分布。
- RL vs DPO: RL 先学 reward model 再优化 policy；DPO 直接用偏好对优化。

易错点

- KD 不一定需要 teacher 参数；很多 LLM KD 只靠 API 输出。
- Teacher 生成数据不一定正确，student 会继承 hallucination、bias、低多样性。
- White-box 方法信息更丰富，但专有强模型通常拿不到内部表示。
- Student 容量、训练数据和目标函数会限制蒸馏上限。

30 秒答题模板

Knowledge distillation 是用 teacher 产生可训练知识，再让 student 学习。流程是确定目标技能和 template，准备 seed examples，让 teacher 生成 distillation data，最后用 SFT、分布对齐、RL 或 DPO 训练 student。Black-box 方法适合 API teacher，white-box 方法可利用 logits/hidden features；主要风险是 teacher 错误和偏见被 student 继承。

14. Reasoning During Decoding

一句话主线：Decoding-time reasoning 不改模型参数，而是在推理时用 CoT、采样投票、搜索、反思或工具提高答案质量。

必背知识点

- Reasoning 指用事实/假设进行结构化思考，得出结论或解释。
- 本讲重点是 inference-time/decoding-time reasoning。
- Chain-of-thought 让模型生成中间推理步骤再回答。
- CoT 局限是线性路径走错后可能一路错下去。
- Self-consistency 采样多条 reasoning chains，对最终答案投票。
- Temperature 控制采样多样性；太高会增加噪声。
- Tree of Thoughts 显式探索 branching reasoning paths。
- ToT 组件：thought decomposition、generator、state evaluator、search algorithm。
- Self-reflection：生成答案 -> 给反馈 -> 修改，可迭代多轮。
- External verifier/solver 调用证明器、代码执行器、符号求解器等验证或求解。

常考对比/流程

- Self-consistency：采样 $r_1, \dots, r_n$ 得到 $a_1, \dots, a_n$ ，选多数答案。
- CoT vs self-consistency vs ToT：单条线性推理 vs 多条完整链投票 vs 树状搜索中间状态。
- ToT 搜索：定义 thought 粒度 -> 生成候选 -> 评价 frontier states -> BFS/DFS 搜索。
- Self-reflection 需要 actionable、specific feedback；笼统反馈无用。
- External verifier 更可靠，但难点是 natural language 到 formal representation。

易错点

- 多数投票不是证明正确；多条链可能共享同一偏差。
- ToT 很贵，而且 problem-specific。
- Self-reflection 不一定提升，模型可能无法发现自己的错。
- 工具不会让模型自动变聪明，模型仍要正确调用和解释工具反馈。

30 秒答题模板

Inference-time reasoning 通过解码策略而不是训练来提升推理。CoT 生成一条推理链；self-consistency 采样多条 CoT 并对答案投票；ToT 把推理变成搜索，包含 thought generator、evaluator 和 BFS/DFS；self-reflection 用具体反馈迭代修改；external verifier 用外部工具检查正确性，但需要正确形式化问题。

15. Mechanistic Interpretability

一句话主线：Mechanistic interpretability 试图 reverse-engineer 模型内部机制，解释 features、neurons、layers、circuits 如何因果产生行为。

必背知识点

- Behavioral evaluation 看输入输出；mechanistic interpretability 看内部计算。
- 方法类型包括 behavioral、attributional、concept-based、mechanistic。
- Feature 是 activation space 中表示某概念/模式的方向。
- Concept 是人类语义概念，可能由多个 features 实现。
- Linear probe 检测信息是否可线性读出，但不能证明模型使用该信息。
- Monosemantic neuron 主要对应一个清晰 feature。
- Polysemantic neuron 对多个不相关 features 激活。
- Superposition 指低维空间用重叠方向编码多于维度数的稀疏 features。
- SAE 从 activations 中学习稀疏 feature dictionary，帮助解开 superposition。
- Circuit 是多个 features/neurons/heads/layers 共同实现某个计算。

常考对比/公式

- Dot product 衡量向量方向对齐，也可看作 feature projection。
- SAE: $c = \text{ReLU}(Mx+b)$ 编码稀疏 feature activations; $\hat{x} = M^T c$ 重构 activation。
- SAE loss = reconstruction loss + L1 sparsity penalty。
- Probe vs SAE: probe 测信息可读出；SAE 学稀疏 feature basis。
- Superposition vs polysemanticity: 前者是表示压缩机制，后者是 neuron 层面的多义现象。
- Universality hypothesis: 不同模型可能学到相似 features/circuits，但必须实证验证。

易错点

- Neuron 不等于 feature；feature 可分布在多个 neuron 上。
- Probe 结果是相关性，不是因果证据。
- SAE 学到的 feature 不一定真实可解释，需要验证。
- Strong universality 不能默认成立，架构、数据、规模都会影响内部机制。

30 秒答题模板

Mechanistic interpretability 目标是解释模型内部如何产生行为，而不只看输入输出。核心概念包括 feature、monosemanticity/polysemanticity、superposition、SAE 和 circuits。Superposition 说明模型用重叠方向压缩许多稀疏 features，导致 neuron 多义；SAE 用稀疏编码学习更可解释的 feature dictionary；最终要用 causal intervention 证明某 circuit 真参与计算。

16. Syntactic Parsing

一句话主线：句法解析把句子变成显式结构，用 constituency 或 dependency 表示词如何组成短语或依存关系，并用 grammar、PCFG 或 neural chart parsing 找到合法高分树。

必背知识点

- Syntax 描述词如何组合成短语和句子，帮助处理结构、歧义和长距离关系。
- Parsing 是为句子构建句法结构，如 constituency tree 或 dependency tree。
- Constituency parsing 关注短语边界和层级，如 NP、VP、S。
- Dependency parsing 关注词与词之间 head-dependent 关系。
- Grammar 要覆盖语言现象，同时避免 overgeneration 和 undergeneration。
- CFG 包含非终结符、终结符、规则和起始符号。
- PCFG 给 CFG 规则加概率，同一左侧规则概率和为 1。
- 一棵 PCFG parse tree 的概率是所有使用规则概率的乘积。
- Supervised parsing 依赖 treebank；unsupervised grammar induction 没有 gold trees，更难。
- Transformer parser 通常先编码句子，再给 spans 或 actions 打分，并用全局解码保证合法树。

常考对比/公式

- Constituency vs Dependency：前者看短语 span 和层级；后者看 head-dependent 边。
- PCFG tree probability: $P(\text{tree}) = \prod \text{prod_rules} P(\text{rule})$ 。
- Parsing evaluation: constituent precision、recall、F1；dependency 常用 UAS/LAS。
- Labelled vs unlabelled constituent evaluation：前者要求 span 和 label 都对；后者只看 span。
- CYK/chart parsing：对每个 span 尝试 split，组合左右子 span 得分，用 backpointers 恢复最佳树。
- Span-based neural parsing：Transformer 编码 $\rightarrow$ 所有 $O(n^2)$ spans 打分 $\rightarrow$ chart decoding 找最高分合法树。

易错点

- Transformer 不会天然输出合法 parse tree，通常还需要 chart parsing 或约束解码。
- 局部 span 高分不代表能组成全局合法树，因为 spans 可能交叉或冲突。
- PCFG 简单可解释，但独立性假设强，难捕捉上下文和长距离依赖。
- Parsing 不只是判断句子是否合法，而是找最可能或最有用的结构。

30 秒答题模板

Syntactic parsing 的目标是给句子建立结构，帮助模型处理短语层级、依存关系和歧义。Constituency parsing 把句子分成嵌套 constituents，如 NP、VP；dependency parsing 则表示词之间的 head-dependent 边。传统方法可用 CFG/PCFG，其中 PCFG 给规则加概率，整棵树概率是规则概率乘积。现代 Transformer parser 用上下文表示给所有 spans 或 actions 打分，但局部高分结构不一定全局合法，所以需要 chart/CYK-style dynamic programming 和 backpointers 找最高分合法树。评价 constituency parsing 常用 precision、recall、F1；dependency parsing 常用 UAS 和 LAS。

17. Perceptually-Grounded NLP

一句话主线：Perceptually-grounded NLP 的核心是：语言意义不只来自文本共现，还要连接到视觉、行动、环境和社会互动。

必背知识点

- Multimodal AI 整合文本、图像、音频、视频等数据来提升理解、预测和决策。
- 本课程重点关注 language 和 vision 的结合，即 Vision+Language Models。
- Text-only representation 只能从文本共现学习意义，缺少直接感知世界、物体属性和行动后果。
- Symbol grounding problem 问的是：如果符号只由其他符号解释，意义如何最终连接到现实 referents。
- Chinese Room argument 说明仅按规则操作符号不等于真正理解意义。
- Chinese/Chinese dictionary-go-round 说明只在符号内部循环会无法落到真实经验。
- Communication grounding 指对话双方共享 mutual knowledge、beliefs、assumptions。
- Perceptual grounding 要让语言系统通过感知和世界互动获得 referents。
- Experience Grounds Language/World Scopes 强调学习者能访问的数据和环境范围决定可学能力。
- Reporting bias 指文本倾向记录值得说的、不寻常的信息，省略显而易见的视觉事实。

常考对比/流程

- Amodal symbols vs grounded symbols：前者只在符号系统内操作；后者与感知、分类和现实 referents 相连。
- Visual grounding 的价值：提供物体、属性、空间关系和动作信号，补充纯文本学习。
- World scope 层级启发：增加模态、行动、互动会改变模型可学习的信息和归纳偏置。
- VLM 任务例子：image captioning、VQA、referring expression、visual dialogue、visual storytelling。

易错点

- 不要说“多模态只是多一个输入格式”；重点是意义、感知和世界经验的连接。
- 扩大 LLM 数据和规模能提升文本能力，但不完全解决 grounding。
- Grounding 有多个含义：communication grounding 和 symbol/perceptual grounding 要分清。

30 秒答题模板

Perceptually-grounded NLP 的动机是 text-only models 只从符号共现学习，受到 symbol grounding problem 和 reporting bias 限制。语言意义与感知、行动、环境和社会互动相关；因此 VLM 用视觉信号把语言表示连接到物体、属性、空间关系和事件。World scopes 进一步说明，学习者能访问的经验范围越丰富，可学到的语言能力和世界知识也不同。

18. Vision-Language Models

一句话主线：VLM 用 pretrained vision encoder、LLM backbone 和 fusion/projector 连接图像与语言；核心考点是架构选择、connector 必要性、训练数据、SmolVLM 和评测。

必背知识点

- VLM 整合视觉和语言数据，使模型能基于图像和文本完成理解、生成和决策。
- 常见架构包括 cross-encoding、cross-attention、visual prefix。
- Cross-encoding 把视觉和语言 tokens 一起输入同一 Transformer。
- Cross-attention 让语言模型通过 attention 访问视觉表示。
- Visual prefix 把视觉表示投影成 LLM 可处理的 prefix tokens，让文本 LLM 在视觉上下文条件下生成。
- Visual prefix 组件：pretrained vision encoder、projector、text-only LLM backbone、image-text training data。
- Vision encoder 常用 ViT 或 CLIP/SigLIP 等视觉-语言 contrastive encoder。
- ViT 把图像切成 patches；patch 数越多，视觉 tokens、context 和显存成本越高。
- VLM 通常在 language tokens 上用 next-token cross-entropy，不对 visual tokens 算 next-token loss。
- VLM 数据包括 noisy web image-text data、captioning、OCR、PDF/charts/tables、multi-image/video、instruction data。

Projector/Connector 必背

- Projector 必要性：vision encoder 输出维度和语义空间不同于 LLM hidden states，需要映射到 LLM 可处理空间。
- Connector 还可能压缩视觉 tokens，控制 context 和 memory cost。
- 选择包括 linear projection、MLP、average pooling、CNN/convolutional mapping、attention pooling/cross-attention with learnable queries。
- Simple connector 便宜稳定但表达力弱；complex connector 更强但成本高、训练更不稳定。
- Compression 提高效率但可能丢失细粒度视觉细节，尤其损害 OCR。
- 训练常先 freeze LLM/vision encoder，只训练 projector；后续再 partial/full finetune，减少 catastrophic forgetting。

SmolVLM 必背

- SmolVLM 使用 SigLIP 视觉 encoder、SmolLM2 language backbone 和 projector。
- Patch 数量公式： $NP=(H/P)^2$ ，例如 224x224 图像、16x16 patch 有 196 patches。
- Pixel shuffle 压缩视觉 token 数，降低空间分辨率、提高深度表示密度，在效率和空间细节间折中。
- SmolVLM 三阶段：adaptation stage 学 projector；image stage 强化图像/OCR/文档理解；video stage 扩展到多图和视频。
- Adaptation stage 通常 freeze language model，使用 noisy web data 和 captioning 数据。
- Image stage 可 finetune 整个模型，并混入文本数据以保留 instruction-following。
- Edge VLM 重要，因为低延迟、隐私、本地部署和成本控制都要求小型高效模型。

评测

- Extrinsic evaluation 用具体任务或应用数据集评 VLM 表现。
- Intrinsic evaluation 直接评表示质量，如 attention pattern、brain alignment、representational quality。
- Task-specific evaluation 明确但迁移性有限；multi-task evaluation 覆盖广但成本高、聚合难。
- 例子：FOIL captions、SVO-Probes、Winoground、VLMEvalKit、MMMUI。
- Winoground 测 visio-linguistic compositionality，要求理解词序、关系和组合意义。

易错点

- 不要把 projector 只说成“维度变换”；它还承担语义对齐、token 压缩、训练稳定性影响。
- 压缩视觉 tokens 不总是好，OCR 和细粒度定位需要保留局部信息。
- VLM 不等于完整 real-world multimodal agent；多数 VLM 缺少行动、环境反馈、声音、触觉和长期交互。

30 秒答题模板

Visual prefix VLM 用 vision encoder 产生 visual tokens，经 projector 映射到 LLM hidden space，作为 prefix 条件化文本 LLM，并在 language tokens 上用 cross-entropy 训练。Projector 必要是因为视觉特征和 LLM token space 维度与语义都不同；它可用 linear/MLP/pooling/CNN/attention 实现，在效率、表达力、稳定性和信息保留之间权衡。SmolVLM 体现了这一管线：SigLIP + SmolLM2 + projector，使用 pixel shuffle 控制 token 数，并通过 adaptation、image、video 三阶段训练扩展能力。评测要区分 extrinsic task benchmarks 和 intrinsic representation evaluation。

19. Text-Only AI Agents

一句话主线：AI agent 是通过 sensors 感知环境、通过 actuators 作用环境的系统；LLM agent 则把目标、状态、计划和动作主要表示为文本与工具调用。

必背知识点

- Russell & Norvig 定义：agent 通过 sensors 感知环境，通过 actuators 作用环境。
- 现代 LLM agent 仍是感知、决策、行动，只是环境常是文本、工具、API 或软件系统。
- LLM-based agent 组成：LLM、输入输出文本表示、wrapper/framework、工具、记忆、workflow control。
- Agentic problem-solving process：Get the Mission、Scan the Scene、Think it Through、Take Action、Learn & Get Better。
- Framework != LLM：用户通常与包装 LLM 的 agent framework 交互，而不是裸 LLM。
- Modality gap：非文本数据必须转成 LLM 可用表示，纯文本 LLM 无法直接处理原始图像、声音或 GUI。
- Context length limit 会导致长任务历史、工具结果、文档被截断或成本上升。
- Lack of memory 指 LLM 本身没有真正长期记忆，长期状态需要 RAG、数据库或外部 memory 模拟。
- Action space 是 agent 可选择和执行的动作集合，决定能力、安全和恢复方式。
- Agent reliability 难，因为规划、工具、上下文和环境交互任何环节都可能出错并级联。

Agentic patterns

- Single-call execution：一次 prompt 解决简单任务；简单但不适合复杂多步。
- Prompt chaining：把任务拆成顺序 prompts；模块化、易调试，但会 error propagation。
- Routing：根据输入类型/意图/安全约束选择流程；动态但 router 会成为 failure point。
- Parallelisation：独立子任务并行执行，再 join/merge；快但需要整合、去重、处理冲突。
- Tool use：LLM 决定何时调用外部函数、数据库或 API，并处理 observation/result。
- MCP 是 Model Context Protocol，用 client-server model 标准化 LLM 与 tools/resources/prompts 的通信。

常考对比/流程

- Tool use 流程：定义工具 -> LLM 生成结构化调用 -> 执行工具 -> observation 回到模型 -> 最终回答或继续调用。
- Tool definition 包含名称、功能描述、参数 schema、调用约束。
- Routing 可用 rule-based、classifier-based、embedding-based、LLM-based。
- Tool use vs RAG：RAG 是检索外部信息的特定模式；tool use 更广，可调用数据库、API、代码、搜索或 RAG。

易错点

- 很多“agent”只是 LLM wrapper，不要被 hype 混淆；要看是否有状态、动作、反馈和环境交互。
- Chaining 提高结构化，但早期错误会传给后续步骤。
- Tool use 扩展能力，也带来结构化输出错误、工具执行失败和权限安全风险。
- Agent 能执行真实动作时必须控制 action space、权限和参数验证。

30 秒答题模板

AI agent 可定义为通过 sensors 感知环境、通过 actuators 作用环境的系统。LLM agent 用语言表示目标、状态、计划和动作，通常由 LLM、wrapper、memory、tools 和 workflow control 组成。常见可靠性模式包括 single-call、prompt chaining、routing、parallelisation 和 tool use：single-call 简单但弱，chaining 可拆解但会传播错误，routing 动态但可能误分类，parallelisation 快但要合并，tool use 扩展能力但增加执行风险。MCP 则是标准化 agent 与外部工具和资源通信的协议。

20. Vision-Based Agents and OpenCUA

一句话主线：Vision-based agents 把 agent 从 text-only 扩展到图像、屏幕和 embodied environment；Computer Use Agent 的核心是用视觉状态、轨迹数据和结构化动作完成长程电脑任务。

必背知识点

- VLM-based agent 输入可包含 vision+text，输出通常仍可用 text/code/action 表示。
- 现实世界包含图像、视频、图表、表格和 GUI，仅靠文本会丢失大量状态信息。
- 视觉输入可避免把 GUI/DOM/图像细节全部转成超长文本，从而缓解 context saturation。
- Embodied AI 是能通过视觉或其他感知流与模拟或真实环境交互的 agents。
- Computer Use Agent 能控制电脑和应用程序，根据语言指令规划并执行点击、输入、滚动等操作。
- Vision-based CUA 的状态通常是当前屏幕截图，而不是 DOM/HTML。
- 轨迹表示： $I, s_0, a_0, s_1, a_1, \dots, s_T, a_T$ 。
- I 是语言 instruction， s_t 是状态/截图， a_t 是动作，如 `click(x,y)`、`type(text)`、`scroll`、`press key`、`terminate`。
- Termination action 重要，因为 agent 需要知道何时停止，否则会循环或执行多余动作。
- 动作表示要结构化，才能被系统可靠执行和评估。

Training data and OpenCUA

- Vision-based agents 需要 trajectories，而不是单纯 image-caption data。
- AgentNet 收集人类电脑操作数据，包括 screen recording、鼠标键盘输入、accessibility trees。
- AgentNet 跨 Ubuntu、Windows、macOS，覆盖多应用和网站，提高跨环境泛化。
- 原始 screen recordings 不能直接训练，因为帧率高、动作密集且低层噪声多。
- Data curation 包括 action reduction 和 state-action matching。
- Action reduction 降低动作密度，只保留高层有用信号。
- State-action matching 从视频中抽取能预测当前动作的关键帧，与动作配对。
- OpenCUA 发现直接用 (I, s_t, a_t) finetune 效果不好，因为模型难以 grounding 和 reasoning over trajectory。
- Long-form CoT for CUA 提供 observation、thought、action 中间监督。
- OpenCUA 三层：L3 observation + thought + action；L2 thought + action；L1 action only。
- OpenCUA 还加入 action history 和 visual history，帮助模型追踪任务进展。
- 训练阶段：Stage 1 GUI screen visual grounding；Stage 2 OS environment planning + reasoning。

Evaluation

- CUA evaluation 核心看 agent 是否能根据语言指令在真实或模拟电脑环境中达到最终目标。
- Success rate 衡量任务成功完成比例，但不解释中间失败。
- 还要评估 visual grounding、循环、backtracking、intermediate goals、failure causes、安全性。
- OSWorld 评估真实电脑环境中的 multimodal agents open-ended tasks。
- OSWorld-Verified 修复网站变化、指令歧义、评估函数脆弱等问题，使评测更可复现。
- WindowsAgentArena、ScienceBoard、Online-Mind2Web、ComputerUseArena 分别测试不同 OS、科学工作流、live websites、human-centric open-ended tasks。
- Future challenges：visual grounding gap、long-horizon modelling、credit assignment、security & reliability。

易错点

- CUA 不是普通搜索 agent；它要连续观察界面、执行动作、处理状态变化和错误恢复。
- 不使用 DOM/HTML 会更接近真实视觉操作，但也更难，因为要从 screenshot 中 grounding。
- Success rate 不足以评价长任务，需要分析失败步骤和安全风险。
- Synthetic CoT 有用，但也可能引入模型教师的偏差和不真实推理。

30 秒答题模板

Computer Use Agent 是能根据语言指令观察屏幕、规划并执行电脑操作的视觉 agent。状态通常是截图 s_t ，动作是点击、输入、滚动、终止等结构化操作，完整任务表示为 trajectory $I, s_0, a_0, \dots, s_T, a_T$ 。OpenCUA 先用 AgentNet 收集和清理人类操作 trajectories，再用 long-form CoT 构造 observation/thought/action 监督，并训练 VLM 做 GUI grounding 与 OS planning/reasoning。CUA 评测困难，因为任务长、环境变化大、成功率不足以解释失败，还要看 grounding、循环、backtracking、credit assignment 和安全可靠性。

最后总速记

- MT: $T^* = \operatorname{argmax}_T p(T|S)$, 平行语料 + encoder-decoder + adequacy/fluency/ChrF/COMET。
- QA: RC 给 passage; ODQA 先 retrieve 后 read/generate; FiD 在 decoder 融合多 passage。
- Data: 来源、筛选、去重、混合、合成和伦理共同决定 LLM 上限。
- Evaluation: 没有 one true metric; use case、ground truth、metric、prompt、deployment criteria 一起定。
- Long context: 位置泛化 + $O(n^2)$ attention + lost-in-the-middle。
- RAG: parametric memory + non-parametric memory; 检索相关性和 attribution 一样重要。
- Multilingual: data mix、tokenizer fertility、transfer/interference、文化适配评价。
- Ethics/Safety: harm taxonomy、bias measurement/mitigation、watermark、jailbreak、adversarial suffix。
- SFT/RLHF: SFT 学格式和行为; RLHF/GRPO 用 reward 优化 policy, 但有 reward hacking。
- Summarisation: 压缩 + 覆盖 + 连贯 + faithfulness; ROUGE/BERTScore 不能完全测事实性。
- PEFT: LoRA 改权重, prompt tuning 改输入, adapters 加函数。
- Distillation: teacher 生成知识, student 用 SFT/divergence/RL/DPO 学。
- Reasoning: CoT、自一致性、ToT、reflection、external verifier 都是 inference-time 策略。
- MI: feature、superposition、SAE、circuits; probe 不是因果证明。
- Parsing: constituency/dependency; PCFG; span scoring + chart/CYK decoding。
- Grounding/VLM: symbol grounding + visual prefix + projector + SmolVLM data/stages + VLM evaluation。
- Agents: agent = sense/decide/act; text-only patterns 到 vision-based CUA, 关键是 trajectories 和 robust evaluation。